A Scalable Python Pipeline for IMU-Based Behavioral Clustering

Chuqi Wang

June 17, 2026

Abstract

This document summarizes the current `Python_Pipeline` implementation for processing Harp IMU recordings and clustering short behavioral windows. The pipeline replaces the original MATLAB workflow with a Python implementation that preserves the key representation used by the prior system: each behavioral window is converted into channel-wise histograms, the histograms are transformed into cumulative distribution functions (CDFs), and distances between windows are computed as a CDF-based approximation to one-dimensional Earth Mover's Distance (EMD), also known as Wasserstein-1 distance. We describe the raw data format, missing-packet handling, signal processing, histogram feature construction, CDF/EMD distance computation, seven Affinity Propagation (AP)-family clustering methods, and the HDBSCAN reference method. We then summarize current comparison results across eight datasets, using silhouette score and agreement with full AP as the main evaluation criteria.

1 Project Goal

This project analyzes mouse behavior from wearable inertial measurement unit (IMU) recordings. The central computational task is to partition continuous motion sensor data into short behavioral windows and assign each window to an unsupervised behavioral cluster. These clusters can then be used for downstream analyses, such as comparing behavioral motif usage across control and Parkinsonian mice or across different arena conditions.

The original MATLAB pipeline performs Affinity Propagation clustering using an EMD-based similarity matrix. However, the MATLAB implementation becomes expensive because it computes many pairwise EMD distances and stores an $N \times N$ similarity matrix, where N is the number of behavioral windows. The Python pipeline keeps the same conceptual distance metric while introducing vectorized preprocessing, CDF-based EMD computation, approximate/exact nearest-neighbor search with FAISS, and several scalable AP variants.

2 Data Overview

2.1 Raw Harp Motion Data

The raw input is a Harp motion CSV file. Rows can have variable width, so `prepare_test_data.py` first pads all rows into a consistent table. Motion packets are identified by:

$$\texttt{RegisterAddress} = 34. \tag{1}$$

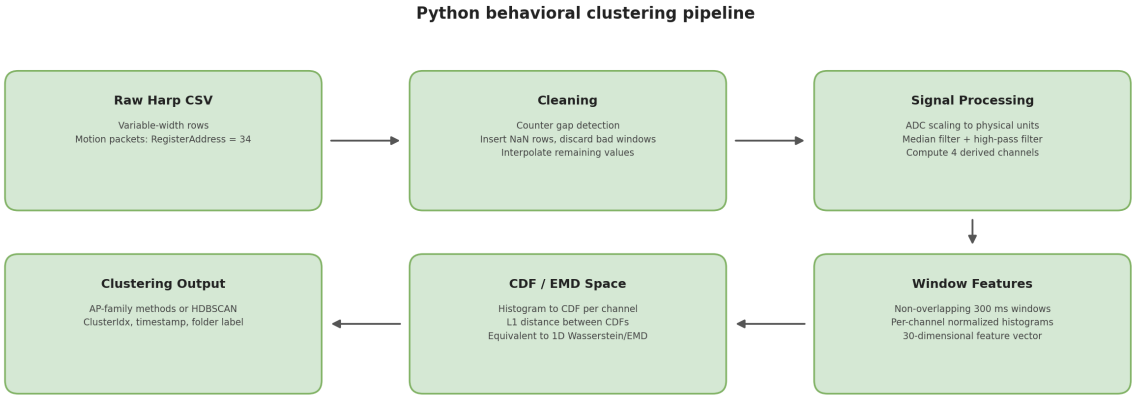

Figure 1: Overview of the Python pipeline from raw Harp CSV files to behavioral cluster labels.

Only these rows contain IMU motion samples. After filtering to motion rows, the Python loader extracts:

$$\texttt{raw\_motion} \in \mathbb{R}^{N_{\text{samples}} \times 10}, \tag{2}$$

with columns

$$\texttt{[ax, ay, az, gx, gy, gz, mx, my, mz, counter]}.$$

The first three columns are accelerometer readings, the next three are gyroscope readings, the next three are magnetometer readings, and the last column is the packet counter. Timestamps and folder/session labels are stored separately. The magnetometer columns (`mx`, `my`, `mz`) are present in the raw data but are not used by the current pipeline.

2.2 Missing-Packet Detection and Cleaning

The packet counter wraps around from 127 to 0. Consecutive motion packets should therefore have counter difference:

$$\Delta c = (c_{t+1} - c_t) \bmod 128 = 1. \tag{3}$$

If $\Delta c > 1$, then $\Delta c - 1$ motion packets were dropped. The preprocessing script inserts placeholder rows with NaN sensor values and linearly interpolated timestamps. This preserves temporal alignment before windowing.

The cleaning rule is window-based. Since one behavioral window contains 60 samples, corresponding to 300 ms at 200 Hz, the script checks missingness inside each 60-row block. If more than 10% of samples in a window contain NaNs in the sensor columns, the entire window is discarded. Remaining NaNs are linearly interpolated. The cleaned motion data are finally trimmed to a multiple of 60 samples.

3 Signal Processing

After loading the cleaned motion rows, `clustering_pipeline.py` applies signal processing designed to mirror the MATLAB pipeline.

3.1 Scaling to Physical Units

Raw accelerometer and gyroscope ADC values are scaled as:

$$\mathbf{a} = \texttt{raw\_accel} \times \frac{4}{32768}, \tag{4}$$

$$\boldsymbol{\omega} = \texttt{raw\_gyro} \times \frac{1000}{32768}. \tag{5}$$

The accelerometer range is $\pm 4g$, and the gyroscope range is ± 1000 degrees per second.

3.2 Filtering and Derived Signals

The pipeline applies a clipped-boundary median filter with kernel size 7. It then applies a zero-phase first-order Butterworth high-pass filter at 0.5 Hz to separate body acceleration from slower gravitational/postural components. The code constructs the four channels used for histogram features:

$$y_{\mathrm{GA}} = y - y_{\mathrm{BA}}, \tag{6}$$

$$z = \text{median-filtered vertical acceleration}, \tag{7}$$

$$z_{\mathrm{gyro}} = \text{median-filtered gyroscope z-axis}, \tag{8}$$

$$a_{\mathrm{tot}} = \sqrt{x_{\mathrm{BA}}^2 + y_{\mathrm{BA}}^2 + z_{\mathrm{BA}}^2}. \tag{9}$$

The fourth histogram channel uses $\log(a_{\mathrm{tot}})$, with a small numerical floor to avoid $\log(0)$.

4 Windowing and Histogram Representation

4.1 Behavioral Windows

The processed time series is split into non-overlapping windows:

$$W_i = \{60 \text{ consecutive samples}\}. \tag{10}$$

At 200 Hz, 60 samples correspond to:

$$\frac{60}{200} = 0.3 \text{ s} = 300 \text{ ms}. \tag{11}$$

Thus, each row of the clustering input represents one 300 ms behavioral segment.

4.2 MATLAB-Compatible Histogram Bins

The current Python code uses bin edges recovered from the MATLAB `hardHistNew.mat` thresholds. The resulting feature vector has 30 dimensions:

$$11 + 11 + 6 + 2 = 30. \tag{12}$$

For a window W_i and channel c, the pipeline counts how many of the 60 samples fall into each bin:

Table 1: Histogram channels and bin counts used by the current Python implementation.

Channel	Number of bins	Meaning
y_{GA}	11	Gravity/postural component of the y-axis acceleration
z	11	Median-filtered z-axis acceleration
z_{gyro}	6	Median-filtered z-axis angular velocity
$\log(a_{\mathrm{tot}})$	2	Log total body acceleration magnitude

$$n_{i,c,b} = \#\{x_t \in W_i : x_t \text{ falls in bin } b\}. \tag{13}$$

The count histogram is normalized by window length:

$$h_{i,c,b} = \frac{n_{i,c,b}}{60}. \tag{14}$$

Therefore, each per-channel histogram is a discrete probability distribution:

$$\sum_b h_{i,c,b} = 1. \tag{15}$$

The four channel histograms are concatenated:

$$\mathbf{h}_i = \left[\mathbf{h}_{i,y_{\mathrm{GA}}}, \mathbf{h}_{i,z}, \mathbf{h}_{i,z_{\mathrm{gyro}}}, \mathbf{h}_{i,\log(a_{\mathrm{tot}})}\right] \in \mathbb{R}^{30}. \tag{16}$$

4.3 Worked Example: Data at Each Processing Stage

Tables 2 and 3 show five consecutive raw samples and their processed equivalents from the `short_comparison_test` dataset (starting at $t \approx 339.47\,\mathrm{s}$). Figure 2 shows the resulting 30-dimensional histogram feature vector for the 300 ms window centred at $t = 339.63\,\mathrm{s}$, chosen as a representative example with values spread across multiple bins in each channel.

Table 2: Stage 1 — raw ADC samples. All ten sensor columns are shown. `ax`–`gz`: accelerometer and gyroscope integer counts; `mx`–`mz`: magnetometer counts (not used by the pipeline); `ctr`: packet counter (wraps 0–127).

t (s)	ax	ay	az	gx	gy	gz	mx	my	mz	ctr
339.473	−7692	−923	1368	3253	930	−897	−30	−18	−34	27
339.478	−8041	−1740	1876	4787	−1121	1091	−30	−18	−34	28
339.483	−8823	−1796	2243	2636	−2255	1893	−30	−18	−34	30
339.488	−8099	−849	2510	404	−2231	1947	−33	−17	−36	31
339.493	−6685	−350	4486	−265	−1887	3228	−33	−17	−36	32

Table 3: Stage 2 — processed signals after ADC scaling, median filtering, and high-pass filtering. Units: y_{GA} and z in g; z_{gyro} in $\mathrm{deg\,s^{-1}}$; $\log(a_{\mathrm{tot}})$ dimensionless.

t (s)	y_{GA}	z	z_{gyro}	$\log(a_{\mathrm{tot}})$
339.473	−0.145	0.306	−27.4	−0.776
339.478	−0.143	0.306	33.3	−0.769
339.483	−0.142	0.306	57.8	−0.763
339.488	−0.141	0.306	59.4	−0.756
339.493	−0.140	0.306	98.5	−0.788

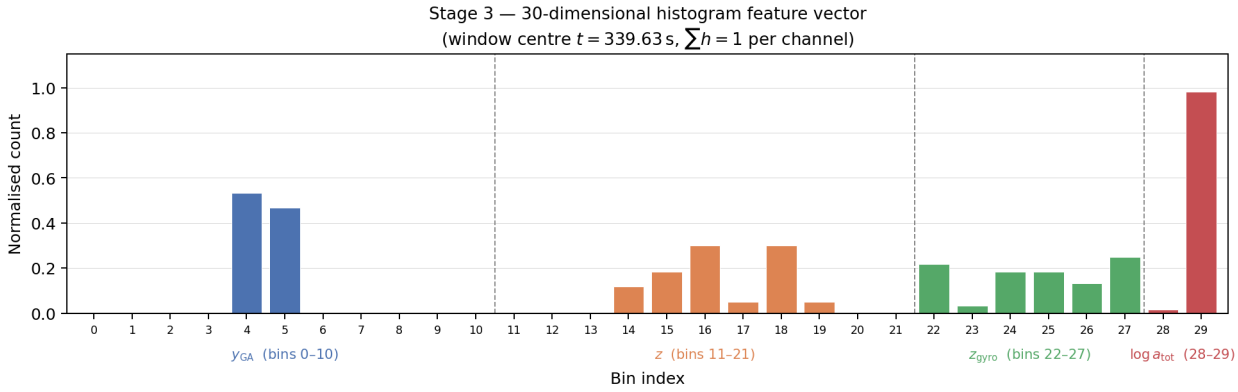

Figure 2: Stage 3 — 30-dimensional normalised histogram feature vector for the 300 ms window centred at $t = 339.63$ s. Each colour corresponds to one sensor channel; dashed lines mark channel boundaries. The y_{GA} channel splits across bins 4–5, indicating the mouse posture oscillated near the boundary between two ranges during this window. The z channel shows a bimodal pattern (bins 16 and 18), and all six z_{gyro} bins are occupied, reflecting varied angular velocity. Each channel sums to 1.

5 CDF Transformation and EMD Distance

5.1 CDF Representation

Earth Mover's Distance measures how much "probability mass" must be moved to transform one distribution into another [4]. In one dimension, Wasserstein-1 distance between two normalized histograms can be computed efficiently as the L1 distance between their cumulative distribution functions (CDFs).

For each channel c, define the CDF of window i as:

$$H_{i,c,b} = \sum_{r=1}^{b} h_{i,c,r}. \tag{17}$$

The Python code additionally divides the CDF L1 sum by $(B_c - 1)$, where B_c is the number of bins for channel c. This gives each channel comparable weight even when it has a different number of bins.

5.2 Per-Channel EMD Approximation

For two windows i and j, the per-channel CDF distance is:

$$d_c(i,j) = \frac{1}{B_c - 1} \sum_{b=1}^{B_c} |H_{i,c,b} - H_{j,c,b}| \,. \tag{18}$$

This is the one-dimensional Wasserstein/EMD distance for uniformly spaced bins, up to the normalization used in the implementation.

5.3 Total Behavioral Distance

The total distance between two behavioral windows is the sum across channels:

$$d(i,j) = \sum_{c=1}^{4} d_c(i,j). \tag{19}$$

In code, the four per-channel CDF vectors are concatenated into a single feature vector $\mathbf{C}_i \in \mathbb{R}^{30}$, so the total distance reduces to a single L1 call:

$$d(i,j) = \|\mathbf{C}_i - \mathbf{C}_j\|_1 \,. \tag{20}$$

The per-channel normalization by $(B_c - 1)$ is absorbed into the concatenated representation, so the single L1 call correctly weights all four channels.

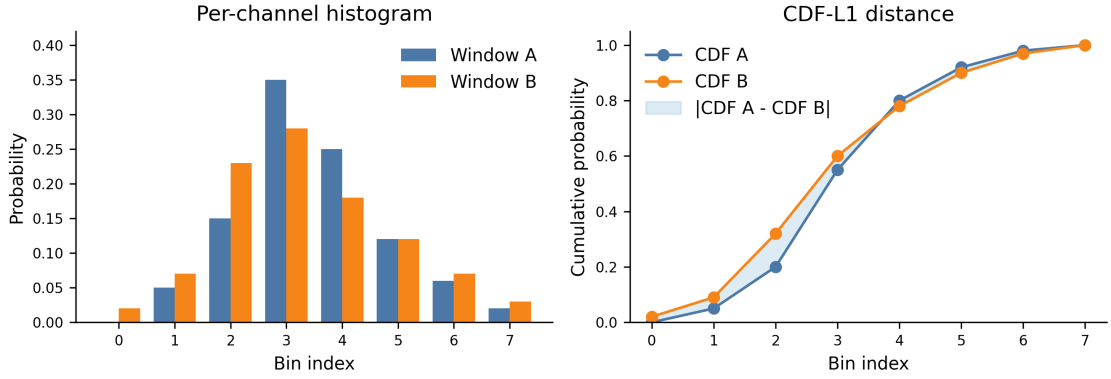

Figure 3: Toy example showing how two window-level histograms are transformed into CDFs. In one dimension, the L1 area between CDFs gives the Wasserstein/EMD distance used by the clustering methods.

5.4 AP Similarity

Affinity Propagation requires similarities rather than distances. The MATLAB pipeline uses negative squared EMD similarity:

$$s(i,j) = -d(i,j)^2. \tag{21}$$

The Python AP methods follow the same similarity convention, but the preference setting depends on the AP variant. For full AP, the preference is set to:

$$p = \min_{i,j} s(i,j), \tag{22}$$

matching the current MATLAB setting `param = min(s(:,3))` in `VPAPPAxes.m`, where column 3 of `s` stores the pairwise similarity values. For random-sampled AP, coreset AP, sparse AP, k-means-sampled AP, and stratified AP, the code estimates the same global minimum similarity by sampling 500,000 random pairs from all N windows. Two-level AP is the main exception: it uses median affinity within local blocks and again among candidate exemplars, because a global minimum preference is too conservative at those smaller local scales.

6 Role of FAISS

FAISS [2] is not itself the behavioral clustering algorithm in this pipeline. It is used as a fast nearest-neighbor engine operating on the CDF feature vectors computed in the previous section. In the AP sampled, coreset, k-means sampled, stratified, sparse, and two-level methods, AP finds a smaller set of exemplars. FAISS then answers:

For this window, which exemplar has the smallest CDF-L1 distance?

Mathematically, if $\mathcal{E} = \{\mathbf{e}_1, \ldots, \mathbf{e}_K\}$ is the final exemplar set, every window i receives label:

$$\ell_i = \arg \min_{k \in \{1,\ldots,K\}} \|\mathbf{C}_i - \mathbf{e}_k\|_1 . \tag{23}$$

Thus FAISS accelerates assignment and KNN graph construction; it does not replace AP's exemplar selection.

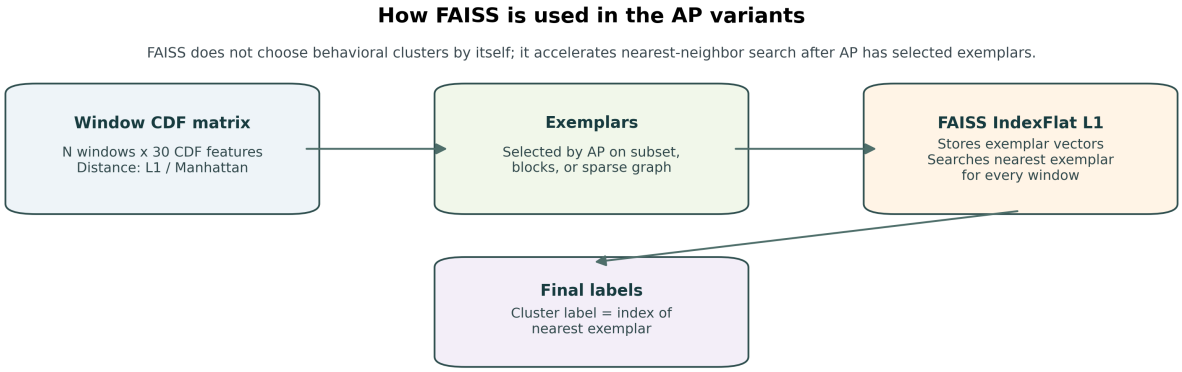

Figure 4: FAISS usage in the AP variants. AP selects exemplars; FAISS stores those exemplar vectors and performs fast nearest-neighbor search under L1 distance to assign every window to its closest exemplar.

7 Why the Python Pipeline Is Faster Than the MATLAB-Style Approach

The speed improvement comes from two separate changes. First, the distance computation is vectorized: each window is represented as a CDF vector, and the EMD-like distance between two

windows is computed by a simple L1 distance between CDF vectors. This avoids repeatedly calling a MATLAB/MEX EMD routine inside nested loops.

Second, the scalable AP variants avoid constructing the full dense $N \times N$ similarity matrix for large datasets. Full AP is still available as a reference method, but the scalable methods run AP on a subset, on local blocks, or on a sparse KNN graph, then use FAISS to assign all windows to the final exemplars. The assignment step scales with the number of windows times the number of exemplars rather than all pairs of windows.

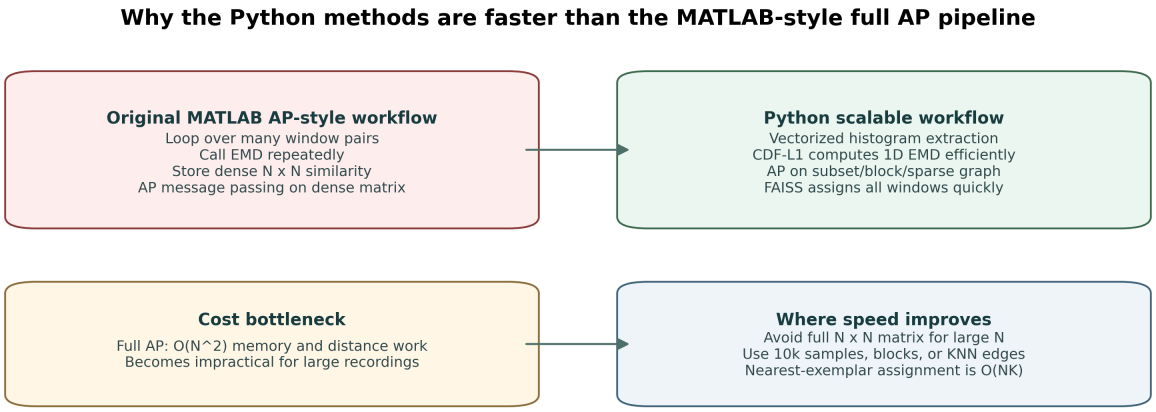

Figure 5: Main computational differences between the MATLAB-style full AP workflow and the scalable Python variants. The largest gain comes from avoiding dense all-pairs AP computations when N is large.

8 Affinity Propagation Algorithm

Affinity Propagation (AP) [1] is an exemplar-based clustering algorithm. Rather than requiring the number of clusters to be specified in advance, AP discovers exemplars — representative data points that best summarize groups of similar points — by iteratively passing messages between all pairs of points until a consistent set of exemplars emerges.

8.1 Problem Formulation

Given N data points with a pairwise similarity matrix $S \in \mathbb{R}^{N \times N}$, AP seeks a subset $\mathcal{E}$ of exemplars and an assignment $e_i \in \mathcal{E}$ for each point i that maximizes the total similarity:

$$\mathcal{E}^* = \arg\max_{\mathcal{E}} \sum_{i=1}^{N} s(i,\, e_i), \tag{24}$$

subject to the self-consistency constraint that every exemplar must represent itself:

$$e_k = k \quad \text{for all } k \in \mathcal{E}. \tag{25}$$

In the current pipeline, the similarity between two windows is the negative squared CDF-L1 distance:

$$s(i,k) = -d(i,k)^2 = -\left\|\mathbf{C}_i - \mathbf{C}_k\right\|_1^2. \tag{26}$$

The diagonal $s(k,k) = p$ is the *preference* parameter, which controls how many exemplars are found. A higher preference (closer to zero) yields more exemplars and therefore more clusters; a lower preference (more negative) yields fewer.

8.2 Message-Passing Equations

AP maintains two $N \times N$ message matrices that are updated alternately at each iteration.

Responsibility $r(i,k)$. The responsibility $r(i,k)$ is sent from point i to candidate exemplar k. It reflects how much point i "prefers" k as its exemplar relative to all other candidates:

$$r(i,k) \;\leftarrow\; s(i,k) - \max_{k' \neq k} \big\{ a(i,k') + s(i,k') \big\}. \tag{27}$$

A large positive $r(i,k)$ means k is a better candidate for i than any alternative.

Availability $a(i,k)$. The availability $a(i,k)$ is sent from candidate exemplar k to point i. It reflects how "willing" k is to be an exemplar, given the support it receives from other points.

For off-diagonal entries ($i \neq k$):

$$a(i,k) \;\leftarrow\; \min\left(0,\;\; r(k,k) + \sum_{i' \notin \{i,k\}} \max\big(0,\, r(i',k)\big)\right). \tag{28}$$

The minimum with zero enforces that only already-supported candidates can attract more points. For the self-availability ($i = k$):

$$a(k,k) \;\leftarrow\; \sum_{i' \neq k} \max\big(0,\, r(i',k)\big). \tag{29}$$

Damping. To prevent numerical oscillations during message passing, each new message is mixed with its previous value using damping factor $\lambda \in [0.5, 1)$:

$$r \;\leftarrow\; (1-\lambda)\, r_{\text{new}} + \lambda\, r_{\text{old}}, \tag{30}$$

$$a \;\leftarrow\; (1-\lambda)\, a_{\text{new}} + \lambda\, a_{\text{old}}. \tag{31}$$

The current pipeline uses $\lambda = 0.9$ for all AP variants (both the scikit-learn calls and the custom sparse AP implementation). This is higher than the scikit-learn default of 0.5, and was chosen to improve convergence stability on the high-dimensional CDF feature space.

8.3 Convergence and Assignment

At each iteration, the *criterion matrix* $C = R + A$ is used to determine exemplars. Point k is identified as an exemplar when:

$$r(k,k) + a(k,k) > 0. \tag{32}$$

The algorithm converges when the set of exemplars remains unchanged for a fixed number of consecutive iterations (default: 15 in scikit-learn) or when the maximum iteration count is reached. After convergence, every point i is assigned to the exemplar that maximises its criterion:

$$e_i = \arg\max_k \, \big\{ r(i,k) + a(i,k) \big\}. \tag{33}$$

8.4 Pseudocode

Algorithm 1 summarizes the full AP procedure.

Algorithm 1 Affinity Propagation

Require: Similarity matrix $S \in \mathbb{R}^{N \times N}$, preference p, damping $\lambda = 0.9$, `max_iter` $T = 1000$, `convergence_iter` $C = 15$
Ensure: Exemplar set $\mathcal{E}$, assignment labels $\{e_i\}$

$S[k,k] \leftarrow p$ **for all** k ▷ set diagonal preference
$R \leftarrow \mathbf{0}_{N \times N}$, $A \leftarrow \mathbf{0}_{N \times N}$ ▷ initialize messages
stable $\leftarrow 0$
for $t = 1$ **to** T **do**
  // Update responsibilities
  for each i **do**
    $\mathrm{AS}[k] \leftarrow A[i,k] + S[i,k]$ **for all** k
    $R_{\text{new}}[i,k] \leftarrow S[i,k] - \max_{k' \neq k} \mathrm{AS}[k']$ **for all** k
  end for
  $R \leftarrow (1-\lambda)\, R_{\text{new}} + \lambda\, R$
  // Update availabilities
  for each k **do**
    $\mathrm{Rp}[i'] \leftarrow \max(0,\, R[i',k])$ **for all** $i' \neq k$
    $A_{\text{new}}[i,k] \leftarrow \min\!\big(0,\; R[k,k] + \sum_{i' \notin \{i,k\}} \mathrm{Rp}[i']\big)$ **for all** $i \neq k$
    $A_{\text{new}}[k,k] \leftarrow \sum_{i' \neq k} \mathrm{Rp}[i']$
  end for
  $A \leftarrow (1-\lambda)\, A_{\text{new}} + \lambda\, A$
  // Check convergence
  $\mathcal{E} \leftarrow \{k : R[k,k] + A[k,k] > 0\}$
  if $\mathcal{E}$ unchanged from previous iteration **then**
    stable $\leftarrow$ stable $+ 1$
  else
    stable $\leftarrow 0$
  end if
  if stable $\geq C$ **then break**
  end if
end for
$e_i \leftarrow \arg\max_k \{R[i,k] + A[i,k]\}$ **for all** i ▷ final assignment
return $\mathcal{E}$, $\{e_i\}$

8.5 Time and Memory Complexity

Full AP requires storing and updating two $N \times N$ message matrices, giving:

Resource	Cost
Memory	$O(N^2)$
Time per iteration	$O(N^2)$
Total time	$O(T \cdot N^2)$

For $N = 20{,}000$ and $T \approx 200$ iterations, full AP requires approximately 12 GB of memory and several minutes of compute. This is the fundamental bottleneck that motivates all scalable AP variants described in Section 9.

8.6 Parameter Settings Used in This Pipeline

Table 4 lists the AP hyperparameters used across all variants in the current implementation.

Table 4: AP hyperparameter settings used in the current pipeline. Two-level AP uses different iteration limits at each level because block-level AP operates on smaller subproblems.

Method	λ	`max_iter`	`conv_iter`	Preference
AP Full	0.9	1000	15	exact min of $N \times N$ affinity
AP Sampled	0.9	1000	15	global min†
AP Coreset	0.9	1000	15	global min†
AP K-Means	0.9	1000	15	global min†
AP Stratified	0.9	1000	15	global min†
AP Sparse (custom)	0.9	1000	15	global min†
AP Two-level (L1)	0.9	300	15	per-block median affinity
AP Two-level (L2)	0.9	500	30	median among candidate exemplars

† Estimated from 500,000 random pairs drawn from all N windows.

All variants use $\lambda = 0.9$, which is higher than the scikit-learn default of 0.5. A higher damping value slows convergence but reduces oscillation, which improves stability when running AP on high-dimensional CDF feature vectors.

9 Clustering Methods

The current comparison script evaluates seven AP-family methods plus one HDBSCAN reference method. AP methods produce cluster exemplars; HDBSCAN identifies dense regions and may label sparse points as noise.

9.1 Full Affinity Propagation

Full AP [1] is the closest Python equivalent to the MATLAB algorithm. It computes the full pairwise distance matrix:

$$D \in \mathbb{R}^{N \times N}, \tag{34}$$

converts it to a similarity matrix $S = -D^2$, fills the diagonal with the AP preference, and runs scikit-learn's [7] `AffinityPropagation` with precomputed affinities. The preference is set to the exact minimum of the $N \times N$ affinity matrix, matching the MATLAB `AccelCluster` setting. It is the most faithful reference method, but requires $O(N^2)$ memory and $O(N^2)$ work per AP iteration, so it is skipped for datasets with $N > 20{,}000$ in the batch comparison.

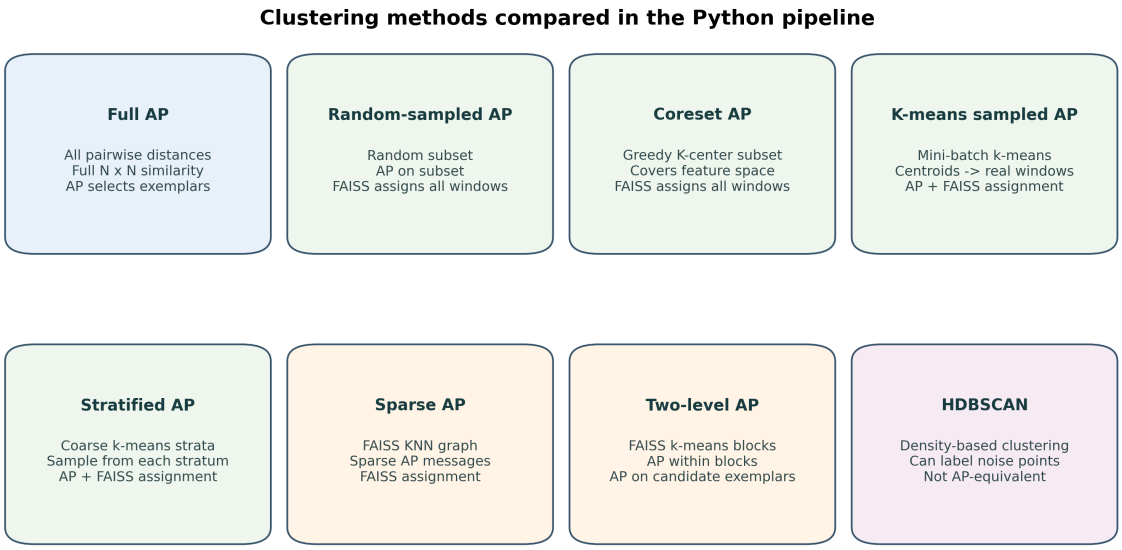

Figure 6: Visual summary of the clustering methods currently compared in `Python_Pipeline`. The AP-family methods differ mainly in how they reduce the cost of exemplar selection before assigning all windows to final exemplars.

9.2 Random-Sampled AP

Random-sampled AP reduces the AP cost by selecting a subset of windows:

1. Randomly sample **10,000 windows**.
2. Run full AP on the sampled windows.
3. Extract AP exemplars from the sample.
4. Use FAISS exact L1 search to assign every original window to its nearest exemplar.

The preference is estimated from 500,000 random pairs drawn from *all* N windows (not just the 10,000 sample), keeping the preference scale consistent with full AP. This is the default scalable AP method in the current pipeline.

9.3 Coreset-Sampled AP

Coreset-sampled AP replaces random sampling with Greedy K-Center selection. The goal is to select representative windows that cover the feature space, including rare behaviors that random sampling may miss.

1. Pick an initial point.
2. Iteratively add the point farthest from the selected set until **10,000 windows** are selected.
3. Run AP on the selected coreset.

4. Assign all windows to the nearest exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). This method can improve coverage of rare behavioral modes, but it may not always best match full AP.

9.4 Sparse AP

Sparse AP avoids the dense $N \times N$ similarity matrix by constructing a K-nearest-neighbor graph:

1. Use FAISS IVFFlat approximate nearest-neighbor search to find K neighbors per point in CDF feature space.
2. Keep similarities only for edges in the KNN graph.
3. Symmetrize the graph and add diagonal preference values.
4. Run custom sparse AP message passing.
5. Assign all windows to the nearest exemplar using FAISS exact L1 search.

The memory cost is approximately $O(NK)$ rather than $O(N^2)$. The current pipeline uses $K = 100$ nearest neighbours by default. However, because the graph is sparse, the result can differ substantially from full AP if K or preference is not well tuned.

9.5 Two-Level AP

Two-level AP is designed to let all windows participate without forming a global dense affinity matrix.

1. Use FAISS k-means to partition all N windows into $M = \max(4, \lfloor N/2500 \rfloor)$ blocks of ∼2,500 windows each (FAISS k-means, `niter`=20).
2. Run full AP inside each block (`max_iter`=300, `conv_iter`=15, preference = per-block median affinity) to obtain local candidate exemplars.
3. Run a second AP over all candidate exemplars M_{cand} (`max_iter`=500, `conv_iter`=30, preference = median affinity among candidates).
4. Assign all windows to the nearest final exemplar using FAISS exact L1 search.

This method reduces AP cost from roughly $O(N^2)$ to approximately $O(N^2/M + M_{\text{cand}}^2)$ over M blocks, where M_{cand} is the number of candidate exemplars collected from all blocks. The median preference at both levels is an intentional design choice: using the global minimum preference would collapse each local block to a single exemplar, leaving too few candidates for the second-level AP.

9.6 K-Means-Sampled AP

K-means-sampled AP uses mini-batch k-means to select central representative windows:

1. Run mini-batch k-means with $k = 10{,}000$ to partition all N windows (`n_init`=3, `batch_size`=min(4096, N), `max_iter`=100).
2. Map each centroid to the nearest real window using FAISS exact L1 search.
3. Run AP on the 10,000 selected real windows.
4. Assign all windows to the nearest AP exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). Compared with random sampling, this method selects cluster-central rather than boundary points, which can improve ARI against full AP.

9.7 Stratified-Sampled AP

Stratified AP first partitions the full dataset into coarse strata and then samples from each stratum.

1. Run mini-batch k-means with $k = 200$ coarse strata (`n_init`=3, `batch_size`=min(4096, N), `max_iter`=100).
2. Draw approximately equal numbers of windows from each stratum until **10,000 windows** are selected in total ($\sim$50 per stratum).
3. Run AP on the stratified sample.
4. Assign all windows to the nearest exemplar using FAISS.

The preference uses the same global min estimate as random-sampled AP (500,000 random pairs from all N windows). By sampling equally from each stratum, this method ensures that rare behaviors occupying small fractions of the recording are represented in the AP sample, regardless of their frequency.

9.8 HDBSCAN Reference Method

HDBSCAN [3] clusters CDF features directly using L1 distance. Unlike AP, it does not require an exemplar preference value and can label low-density windows as noise (assigned label -1). The output can be useful for exploratory density analysis, but it is not expected to match MATLAB AP results closely because it optimizes a different clustering objective.

10 Evaluation Metrics

10.1 Number of Clusters and Noise

For AP methods, every window is assigned to a cluster, so the noise percentage is 0%. For HDBSCAN, some windows may be labeled as noise and are excluded from cluster-size statistics.

10.2 Silhouette Score

Silhouette score [5] measures how close a point is to its own cluster compared with other clusters. The code computes silhouette using L1 distance on CDF features, subsampling up to 2,000 valid windows for speed. Note that HDBSCAN silhouette scores are not directly comparable to AP-family scores: HDBSCAN may achieve high silhouette by labeling ambiguous boundary windows as noise and retaining only well-separated points, whereas AP assigns every window to a cluster. The interpretation used in the report is:

Silhouette range	Interpretation
> 0.5	good
0.25–0.5	reasonable
< 0.25	poor

10.3 ARI Against Full AP

Adjusted Rand Index (ARI) [6] is used when full AP is available. Full AP is treated as the MATLAB-like reference. ARI is interpreted as:

ARI value	Meaning
1	identical clustering
0	agreement expected by chance
< 0	worse than chance

11 Datasets

The current batch comparison evaluates eight datasets.

Table 5: Datasets included in the current method comparison. Duration is computed as $N \times 300$ ms.

Dataset	N	Duration	Description
`short_comparison_test`	1,195	∼6 min	Short subset of comparison_test for quick testing
`comparison_test`	13,799	∼69 min	Full MATLAB-equivalent reference recording
`mp_mouse_1_jul`	19,714	∼99 min	MitoPark Parkinson's mouse, July session
`control_mouse_1_jul`	23,902	∼120 min	Control mouse, July session (age-matched)
`control_mouse_1_oct`	23,906	∼120 min	Control mouse, October session (age-matched)
`mp_mouse_1_oct`	23,866	∼119 min	MitoPark Parkinson's mouse, October session
`still_test`	23,877	∼119 min	Low-motion baseline recording
`moving_test`	47,028	∼235 min	High-motion activity recording

12 Results

12.1 Cluster Counts and Silhouette Scores

Tables 6 and 7 summarize the current cached comparison results. Cluster counts and silhouette scores are shown separately for readability. Full AP is skipped when the dataset has more than 20,000 windows. The accompanying figures use abbreviated axis labels from the plotting code (e.g., *AP twolevel*, *AP kmeans*, *AP stratified*), which correspond to the full method names used in the tables and text.

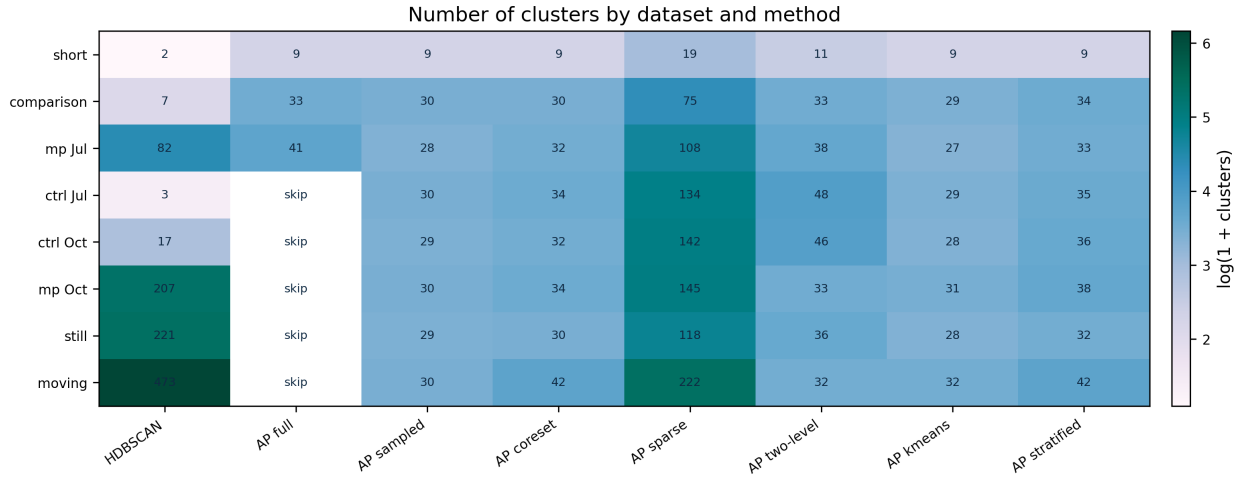

Figure 7: Heatmap of the number of clusters found by each method. Sparse AP and HDBSCAN tend to produce many more clusters than the AP sampled family.

Table 6: Number of clusters found by each method. A dash indicates that AP Full was skipped because the dataset exceeded the full-AP size threshold. High cluster counts in Sparse AP and HDBSCAN reflect known limitations of those methods rather than desirable results.

Dataset	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
short_comparison_test	2	9	9	9	19	11	9	9
comparison_test	7	33	30	30	75	33	29	34
mp_mouse_1_jul	82	41	28	32	108	38	27	33
control_mouse_1_jul	3	–	30	34	134	48	29	35
control_mouse_1_oct	17	–	29	32	142	46	28	36
mp_mouse_1_oct	207	–	30	34	145	33	31	38
still_test	221	–	29	30	118	36	28	32
moving_test	473	–	30	42	222	32	32	42

12.2 Agreement with Full AP

Full AP is available for three datasets. Table 8 reports ARI against full AP. The strongest agreement is usually achieved by sampled AP variants, especially random, k-means, and stratified sampling.

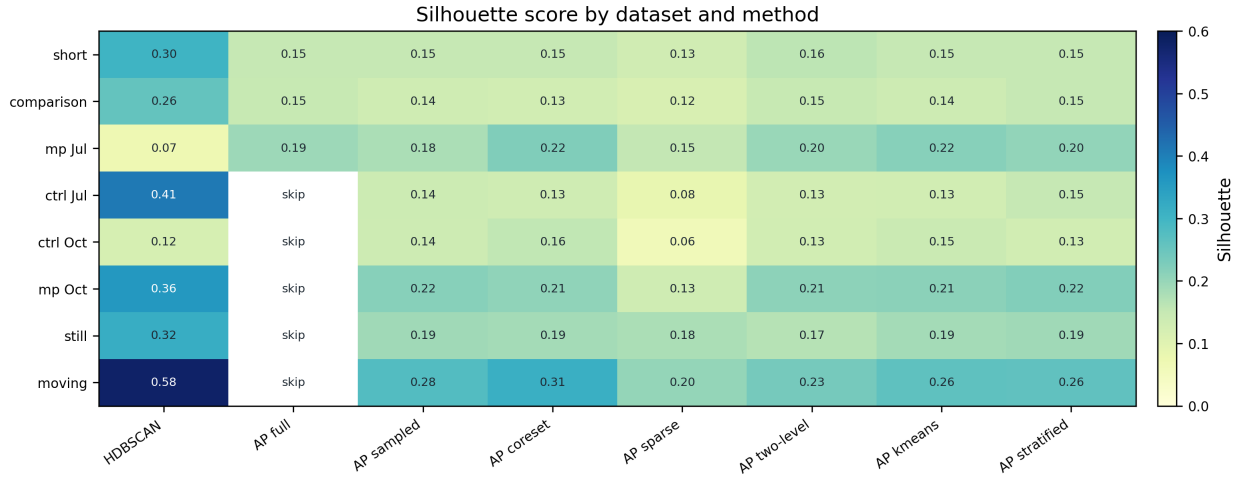

Figure 8: Silhouette score by dataset and method. Higher values indicate stronger geometric separation in the CDF-L1 feature space, but silhouette alone does not imply biological validity.

Table 7: Silhouette score by method. Bold indicates the highest value per row. A dash indicates that AP Full was skipped. HDBSCAN scores are not directly comparable with AP scores; see Section 10.2.

Dataset	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
short_comparison_test	**0.302**	0.152	0.152	0.152	0.133	0.160	0.152	0.152
comparison_test	**0.257**	0.148	0.136	0.129	0.117	0.149	0.138	0.151
mp_mouse_1_jul	0.071	0.193	0.179	**0.225**	0.153	0.197	0.218	0.204
control_mouse_1_jul	**0.408**	–	0.139	0.135	0.085	0.128	0.131	0.151
control_mouse_1_oct	0.121	–	0.138	**0.155**	0.059	0.130	0.145	0.130
mp_mouse_1_oct	**0.359**	–	0.217	0.207	0.135	0.209	0.212	0.221
still_test	**0.317**	–	0.192	0.186	0.179	0.168	0.186	0.190
moving_test	**0.580**	–	0.279	0.313	0.203	0.235	0.265	0.261

The same column abbreviations are used as above.

12.3 Runtime

Table 9 reports wall-clock runtime on a single CPU core. AP Full is the slowest method and becomes impractical beyond $N \approx 20{,}000$. Among scalable methods, HDBSCAN and AP Two-level are the fastest; AP Sampled and AP Coreset are moderate; AP Sparse is the slowest due to the large approximate KNN graph construction step. Timing data for AP K-Means and AP Stratified was not recorded in the current batch run and is omitted from the table.

12.4 Main Observations

1. **Full AP is the reference but does not scale.** It closely matches the MATLAB objective but requires dense pairwise affinities, so it is only practical for small or medium datasets.

Table 8: ARI against full AP for datasets where full AP was run. AP full is omitted because it is the reference method.

Dataset	HDBSCAN	AP Sampled	AP Coreset	AP Sparse	AP Two-level	AP K-Means	AP Stratified
`short_comparison_test`	0.001	**1.000**	**1.000**	0.395	0.548	**1.000**	**1.000**
`comparison_test`	0.004	0.425	0.403	0.351	0.348	**0.458**	0.444
`mp_mouse_1_jul`	0.074	**0.475**	0.423	0.389	0.437	0.459	0.462

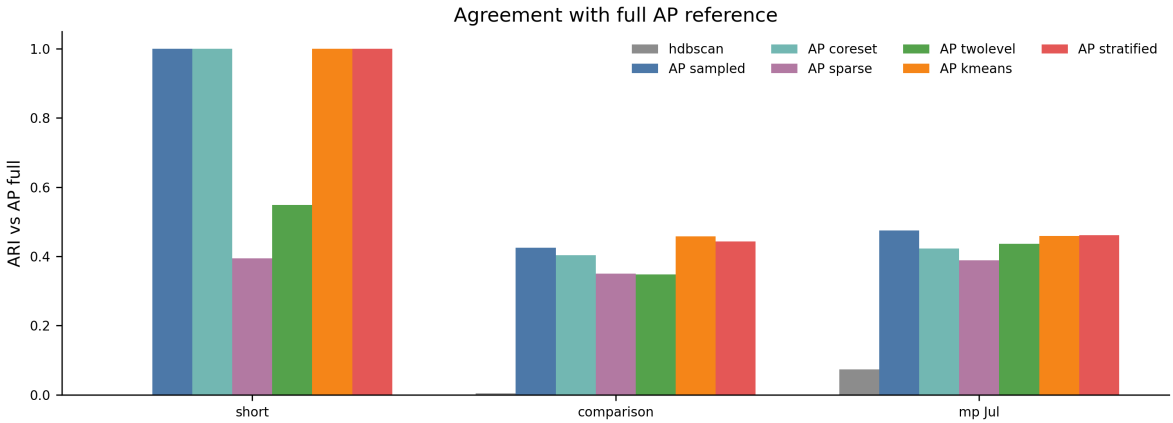

Figure 9: Agreement with full AP measured by ARI. HDBSCAN has low agreement with full AP because it optimizes a density-based objective rather than an exemplar-based AP objective.

2. **Random-sampled AP is a strong default.** It is much faster than full AP and has the highest or near-highest ARI against full AP in the available comparisons.

3. **K-means and stratified sampling are promising.** These methods often match or exceed random sampling in ARI, suggesting that sample selection matters.

4. **Coreset sampling can improve silhouette.** On `mp_mouse_1_jul` and `moving_test`, coreset AP gives higher silhouette than random sampled AP, likely because it covers the feature space more broadly.

5. **Sparse AP currently over-fragments.** It often produces many more clusters than full AP or sampled AP, suggesting that the KNN graph degree and preference should be tuned further.

6. **HDBSCAN is not a MATLAB replacement.** It sometimes has high silhouette, especially in moving/still tests, but its ARI against full AP is very low and it labels many points as noise.

13 Discussion

13.1 Recommended Current Method

For replacing the MATLAB AP pipeline while keeping similar behavior, the best current default is random-sampled AP. It is fast (18–51 s across all datasets in Table 9) and achieves ARI of 1.000,

Table 9: Wall-clock runtime per dataset and method on a single CPU core. A dash indicates AP Full was skipped. The $\sim$ symbol denotes approximate timing for AP Full, which was not run in the main batch.

Dataset	N	HDBSCAN	AP Full	AP Sampled	AP Coreset	AP Two-level	AP Sparse
short_comparison_test	1,195	<0.1 s	0.6 s	0.3 s	0.2 s	0.2 s	0.3 s
comparison_test	13,799	2.3 s	∼135 s	30 s	25 s	17 s	16 s
mp_mouse_1_jul	19,714	1.7 s	∼390 s	33 s	29 s	26 s	61 s
control_mouse_1_jul	23,902	3.9 s	–	18 s	26 s	29 s	81 s
control_mouse_1_oct	23,906	2.8 s	–	20 s	25 s	35 s	113 s
mp_mouse_1_oct	23,866	1.7 s	–	34 s	26 s	28 s	190 s
still_test	23,877	1.7 s	–	24 s	31 s	26 s	54 s
moving_test	47,028	5.7 s	–	51 s	35 s	71 s	172 s

0.425, and 0.475 on the three datasets where full AP is available (Table 8). K-means-sampled AP is a strong alternative: it achieves the highest ARI on `comparison_test` (0.458 vs. 0.425 for random), and stratified AP performs similarly. Full AP should be retained as the reference method for datasets small enough to run it.

13.2 Implications for Parkinsonian Behavior Analysis

For Parkinson's disease progression analysis, rare behavioral modes may be scientifically important. Random sampling can underrepresent rare windows, especially if tremor-like or impaired-movement motifs occupy a small fraction of the recording. For this reason, coreset and stratified sampling are especially relevant. They are not merely speed optimizations; they also change which parts of the behavioral space are represented during exemplar selection.

13.3 Limitations

The current representation intentionally discards temporal ordering inside each 300 ms window. Two windows with the same amplitude distribution but different within-window temporal sequence will have identical histograms. This may be limiting for tremor or rhythmic gait features. In addition, fixed MATLAB-derived bin edges may not be optimal for MitoPark mice if disease changes the dynamic range of motion. Finally, silhouette score is only a geometric clustering measure; it does not directly validate biological interpretability.

14 Conclusion

The current `Python_Pipeline` reproduces the core MATLAB representation using 300 ms windows, channel-wise histograms, CDF-transformed features, and EMD-like distances. The main computational improvement is to avoid full dense $N \times N$ AP whenever possible. Among scalable AP variants, random-sampled AP currently provides the strongest practical balance between speed and agreement with full AP, while k-means and stratified sampling are strong candidates for future refinement. HDBSCAN remains useful as an exploratory density-based baseline but should not be interpreted as a direct replacement for the MATLAB AP workflow.

Implementation Notes

This document is based on the current local contents of:

- `Python_Pipeline/scripts/prepare_test_data.py`
- `Python_Pipeline/scripts/clustering_pipeline.py`
- `Python_Pipeline/scripts/batch_compare.py`
- `Python_Pipeline/scripts/compare_results.py`
- `Python_Pipeline/results/method_comparison.csv`
- `Python_Pipeline/results/method_comparison_report.txt`

References

[1] Frey, B. J. and Dueck, D. (2007). Clustering by passing messages between data points. *Science*, 315(5814):972–976.

[2] Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.

[3] Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 160–172.

[4] Rubner, Y., Tomasi, C., and Guibas, L. J. (2000). The earth mover's distance as a metric for image retrieval. *International Journal of Computer Vision*, 40(2):99–121.

[5] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.

[6] Hubert, L. and Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1):193–218.

[7] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.